

CSC 463

Embedded Systems

Lecture 3: Programming Models & Instruction Sets

Embedded System

Part 3: Programming Models & Instruction Sets

Topics Covered

- ❑ Microprocessor programming models: Intel 80x86 and other selected processors.
- ❑ Instruction set architecture and assembly language programming.
- ❑ Programming tools and environments for embedded system development.

Introduction

- Embedded systems rely heavily on programming models and instruction sets to interact with hardware and execute software instructions efficiently.
- Understanding microprocessor programming models and instruction set architectures (ISA) is critical for designing, implementing, and troubleshooting embedded systems.
- Key aspects include the architecture of processors like Intel 80x86, assembly language programming, and the tools used for embedded system development.

Microprocessor Programming Models

1. Intel 80x86 Programming Model:

- **Features:**

- **Registers:** Includes general-purpose registers (AX, BX, CX, DX), segment registers, and special-purpose registers like instruction pointers and flags.
- **Buses:** Three types—data, address, and control buses.

2. Instruction Execution Model:

- Instructions are executed sequentially unless altered by control flow instructions (e.g., jumps, calls).

Microprocessor Programming Models

- **Addressing Modes:**

- Immediate, direct, register, indexed, and indirect addressing.
- Provides flexibility in accessing and manipulating memory or registers.

3. **Other Selected Processors:**

- Comparisons with non-Intel architectures:
 - Differences in register sets, instruction sets, and addressing modes.
 - Special features unique to specific processors (e.g., ARM's RISC architecture).

Instruction Set Architecture (ISA)

- **The Instruction Set Architecture (ISA)** is a critical part of processor design that serves as the interface between software and hardware. It defines how a processor understands and executes instructions.
- The ISA dictates the types of instructions a processor can execute, how those instructions are formatted, and the processor's interaction with memory and input/output devices.

Key Components of ISA

- **Data Types:**

- The types of data that the processor can handle, such as:
 - **Integer:** Signed and unsigned integers.
 - **Floating-point:** For precise calculations in scientific and engineering applications.
 - **Characters:** ASCII or Unicode values.
 - **Custom types:** Specific to applications, e.g., fixed-point for signal processing.

Instruction Categories

- **Arithmetic and Logical Instructions:**
 - Perform operations like addition, subtraction, multiplication, and bitwise logic (AND, OR, XOR).
- **Data Transfer Instructions:**
 - Move data between registers, memory, and I/O ports (e.g., MOV, LOAD, STORE).
- **Control Instructions:**
 - Manage program flow using branching and looping instructions (e.g., JMP, CALL, RET).

Instruction Categories

- **System Instructions:**
 - Enable interaction with the processor's system-level features like interrupt handling or power management.

Addressing Modes

➤ Methods for accessing data stored in memory or registers:

1. **Immediate:** The operand is part of the instruction itself.
2. **Direct:** Specifies the memory address of the data.
3. **Indirect:** Uses a register to point to the memory address.
4. **Indexed:** Combines a base address with an offset.
5. **Relative:** Address is calculated relative to the current instruction pointer.

Addressing Modes

➤ These modes allow flexibility in programming and optimize memory access.

- **Control Flow:**

- **Branching:** Directs execution to different parts of the program based on conditions (e.g., JNZ for "jump if not zero").
- **Loops:** Repeats blocks of instructions (e.g., LOOP with counter-based iteration).
- **Interrupts:** Temporarily halts program execution to handle external events.

Assembly Language Programming

- **Assembly language** is a human-readable representation of the machine language specific to a processor's ISA. It provides low-level control over the hardware while being easier to understand and write compared to binary code.

Key Features:

- **Mnemonics:**
 - Symbolic names for instructions (e.g., ADD for addition, SUB for subtraction).
 - Helps programmers focus on operations instead of raw binary or hexadecimal codes.

Assembly Language Programming

- **Syntax and Structure:**
 - **Labels:** Named locations in code for marking jump or call destinations.
 - **Operands:** Specify the data to be operated on (e.g., registers, memory addresses).
 - **Directives:** Provide instructions to the assembler (e.g., .DATA to define a data section).

Assembly Language Programming

- **Registers:**

- Assembly often involves direct manipulation of processor registers.

- Example (Intel 80x86):

- General-purpose registers (e.g., AX, BX).
 - Segment registers (e.g., CS for code segment).
 - Special-purpose registers (e.g., EIP for the instruction pointer).

Advantages of Assembly

- High performance and efficiency, as it allows control of every hardware detail.

Assembly Language Programming

- Useful for time-critical applications like real-time systems.
- Compact code, optimizing memory usage.

Challenges of Assembly:

- Complex and error-prone due to its low-level nature.
- Steep learning curve compared to high-level languages.
- Lack of portability—code written for one processor is not compatible with another.

Assembly Code (Intel 80x86):

section .data

msg db "Hello,World!", 0

section .text

global __start

__start:

mov edx, 13 ; Length of the string

mov ecx, msg ; Address of the string

mov ebx, 1 ; File descriptor (stdout)

mov eax, 4 ; System call number for write

int 0x80 ; Call kernel

mov eax, 1 ; System call number for exit

xor ebx, ebx ; Exit code 0

int 0x80 ; Call kernel

THE END